Operaciones entre variables

Algunas operaciones entre variables

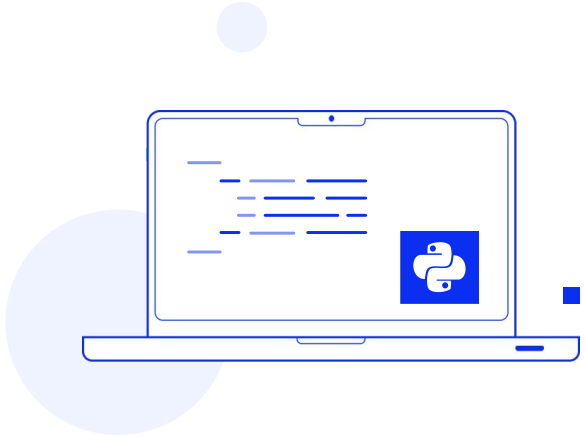
Las **operaciones aritméticas** representan el núcleo más básico de interacción entre variables. **Cada operación tiene sus particularidades**: por ejemplo, la división con el operador **/** siempre devuelve un número de punto flotante, mientras que **//** realiza una división entera.

Python ofrece **operaciones de comparación** que **generan valores booleanos**. Los operadores como mayor que (**>**), menor que (**<**), igual a (**==**), diferente de (**!=**) permiten construir condiciones lógicas fundamentales para el control de flujo en programas.



Los **operadores lógicos and, or y not** posibilitan combinar condiciones, crear estructuras condicionales complejas y tomar decisiones algorítmicas con gran precisión. Estas operaciones son especialmente útiles en estructuras de control como condicionales y bucles.

Python no permite operaciones directas entre tipos incompatibles, demandando **conversiones explícitas mediante funciones como `str()`, `int()` o `float()`** cuando se requiere combinar datos de naturaleza distinta.



Módulo

El módulo (**%**) es un operador aritmético que **devuelve el resto de una división**.

El módulo se revela como un operador versátil, ya que permite:

- Identificar residuos.
- Construir lógicas de paridad.
- Generar secuencias cíclicas.
- Resolver problemas que involucren divisiones y patrones repetitivos.

Veamos el ejemplo del siguiente *slide*.



Ejemplo:

Mostrar por pantalla el resto de una división con valores ingresados por el operador.

```
dividendo = int(input("Escribe el dividendo: "))
divisor = int(input("Ingrese el divisor: "))
resto = dividendo % divisor
print("El resto de la división es", resto)
```

Operadores

Operadores de asignación

Los operadores de asignación permiten **realizar una operación y almacenar su resultado en la variable inicial**.

Los operadores de asignación no son más que **atajos** para escribir de manera más corta, y asignar su resultado a la variable inicial. El operador `+=` en `x += 1` es equivalente a `x = x + 1`.



En Python, no existe el operador `++` (que sí existe en otros lenguajes).

Operador	Ejemplo si x = 7	Equivalente	Resultado
`=`	`x = 7`	`x = 7`	7
`+=`	`x += 2`	`x = x + 2`	9
`-=`	`x -= 2`	`x = x - 2`	5
`*=`	`x *= 2`	`x = x * 2`	14
`/=`	`x /= 2`	`x = x / 2`	3.5
`%=`	`x %= 2`	`x = x % 2`	1
`//=`	`x //= 2`	`x = x // 2`	3
`**=`	`x **= 2`	`x = x ** 2`	49

Asignación compuesta: operador +=

El operador **+=** se utiliza para la **manipulación eficiente de variables numéricas y secuencias**. Este operador de asignación compuesta permite realizar una **suma y asignación** en un solo paso.

Cuando se utiliza **+=** con números enteros o de punto flotante, el operador **incrementa el valor de la variable** de manera directa y concisa. Por ejemplo, si tenemos una variable contador inicializada en 5 y aplicamos contador **+=** 3, el resultado será inmediatamente 8, realizando la operación de suma y actualización de manera simultánea.

En el caso de **secuencias** como listas o cadenas de texto, el operador **+=** funciona como un método de concatenación rápida. En **cadenas**, permite añadir contenido de manera inmediata.



Es importante comprender que este operador no solo **simplifica la sintaxis**, sino que también **puede mejorar el rendimiento** en ciertos contextos, al realizar la operación de manera más directa que los métodos tradicionales de suma y asignación.

Ejemplos

Suma con números enteros:

```
numero = 5
numero += 3  # Equivalente a: numero = numero + 3
print("Suma con enteros:", numero)  # Resultado: 8
```

Concatenación de cadenas:

```
mensaje = "Hola "
mensaje += "mundo"  # Equivalente a: mensaje = mensaje + "mundo"
print("Concatenación de cadenas:", mensaje)  # Resultado: "Hola mundo"
```

X = X + “algo”

El operador **+=** y la asignación tradicional x **=** x **+** `"algo"` pueden parecer similares, pero presentan **diferencias sutiles pero significativas** en su funcionamiento interno y rendimiento.

El **operador +=** realiza una operación de concatenación o suma en el mismo objeto, lo que significa que modifica directamente la referencia de memoria original. En contraste, **x = x + "algo"** **crea un nuevo objeto en memoria**, generando una nueva referencia y potencialmente consumiendo más recursos computacionales.

El **operador += resulta más eficiente** porque evita la creación de objetos intermedios, optimizando el uso de memoria y procesamiento.



```python
# Ejemplo con cadenas
texto1 = "Hola"
texto2 = "Hola"

# Utilizando +=
texto1 += " mundo"  # Modifica directamente el objeto original

# Utilizando asignación tradicional
texto2 = texto2 + " mundo"  # Crea un nuevo objeto

print("Texto con +=:", texto1)
print("Texto con asignación tradicional:", texto2)
```

Otros operadores de asignación compuesta

En Python, existen varios **operadores de asignación compuesta** similares al **+=**, que **combinan una operación aritmética o de otro tipo con la asignación.**

- **Resta y asignación (-=):** Resta el valor de la derecha al de la izquierda y asigna el resultado.

```
x = 10
x -= 3  # x ahora es 7
```

- **Multiplicación y asignación (*=):** Multiplica el valor de la izquierda por el de la derecha y asigna el resultado.

```
y = 4
y *= 2  # y ahora es 8
```

- **División y asignación (/=):** Divide el valor de la izquierda por el de la derecha y asigna el resultado.

```
z = 20
z /= 4  # z ahora es 5.0
```

- **Exponenciación y asignación (**=):** Eleva el valor de la izquierda a la potencia de la derecha.

```
c = 2
c **= 3  # c ahora es 8
```

- **División entera y asignación (//=):** Realiza una división entera y asigna el resultado.

```
a = 17
a //= 5  # a ahora es 3
```

- **Módulo y asignación (%=):** Calcula el resto de la división y asigna el resultado.

```
b = 17
b %= 5  # b ahora es 2
```

Actualización de variables

Modificar el valor

La actualización de una variable implica **modificar el valor almacenado en una variable existente**. Cada vez que se asigna un nuevo valor a una variable, se está efectuando una actualización que **sobrescribe el contenido anterior**.

> Un ejemplo de actualización ocurre en estructuras de conteo o acumulación. Imaginemos un contador que registra la cantidad de participantes en un evento: inicialmente podría comenzar en cero y luego incrementarse cada vez que ingresa un nuevo participante.

Las actualizaciones pueden ser **simples**, como asignar directamente un nuevo valor, o **compuestas**, utilizando operadores que combinan la operación y la asignación.

Cuando se escribe `contador = contador + 1`, se está realizando una actualización que incrementa el valor previo en una unidad. Sin embargo, la versión compacta **`contador += 1`** logra el mismo resultado de manera más elegante.

Es posible actualizar variables mediante **operaciones más complejas**, como concatenaciones en cadenas, transformaciones en listas o aplicación de funciones que modifican el valor original.

Cada actualización genera un nuevo estado para la variable, preservando su identidad pero modificando su contenido. Este concepto resulta fundamental en algoritmos que requieren seguimiento de estados, como contadores, acumuladores o variables de control en ciclos y estructuras de decisión.

